Module 2
Concept of Arrays

2.1 Arrays

- An array is a collection of homogeneous data elements (same data type), stored in contiguous memory locations, and accessed using a common name and index.
- Array elements are accessed using an index that starts from 0 for the first element, 1 for the second, and so on.
- If an array has 'n' elements, the valid indices range from 0 to n-1.

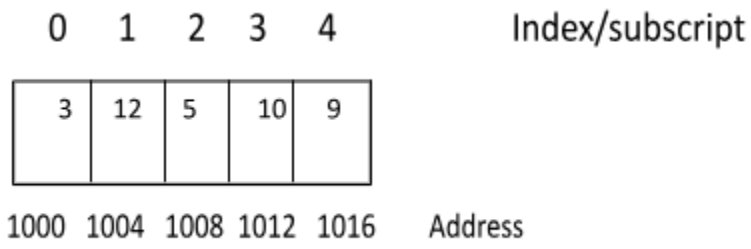


Fig. 2.1 Representation of an array of size 5 in memory

2.1.1 Array Declaration

- Array declaration is the process of specifying the type, name, and size of the array.
- *Syntax:*

  data_type array_name[size];

- *Example:*

  int marks[5];

  This declares an integer array named marks of size 5, i.e., it can store 5 integers.

2.1.2 Initialization of Array

- Array initialization is the process of assigning values to the elements of an array at the time of declaration.
- An array can be initialized either partially or fully using curly braces {}at the time of declaration or later by assigning values to individual elements.
- Syntax is:

  data_type array_name[array_size] = {value1, value2, ..., valueN};

- Initialization can performed in the following ways:
    - int marks[5] = {80, 85, 90, 75, 88}; //Initializes all elements
    - If fewer values are provided, the rest are initialized to 0:
      int marks[5] = {10, 20}; // same as {10, 20, 0, 0, 0}
    - Initialization without size:

      int a[] = {1, 2, 3}; // Compiler determines size = 3
    - Individual Element Assignment:
        - After declaration, you can assign values to elements one by one using their index.

```
int arr[3];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

    - With User Input:
        - Arrays can be populated by taking input from the user, typically within a loop.

```
int arr[5];
printf("Enter 5 integers:\n");
for (int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
}
```

2.1. 3 Accessing Array Elements

- Array in C provides random access to its elements, which means that we can access any element of the array by providing the position of the element
- Array elements can be accessed using their index values.
- The index starts at 0, so the first element is at position 0, the second at position 1, and so on.
- Example:

```
#include <stdio.h>
int main() {
   int arr[5] = {10, 20, 30, 40, 50};  // Declare and initialize an array
   // Accessing elements
   printf("First element: %d\n", arr[0]); //Prints 10
   printf("Third element: %d\n", arr[2]); //Prints 30
   return 0;
}
```

2.1.4 Input and Output Array Elements

- Taking user input for an array.

```
#include <stdio.h>
int main() {
```

```
    int arr[5];
    // Input elements
    printf("Enter 5 integers: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }
    // Output elements
    printf("Entered elements are: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

/* Program to find Sum of Array Elements*/

```
#include <stdio.h>
void main() {
   int arr[5], sum = 0;
   printf("Enter 5 elements:”);
   for (int i = 0; i < 5; i++) {
      scanf("%d",&arr[i]);
   }
   for (int i = 0; i < 5; i++) {
       sum += arr[i];
   }
   printf("Sum = %d", sum);
}
```

2.2 Operations on Array

- Operations on arrays are fundamental actions performed to manage and manipulate the elements stored within an array data structure.
- Some of the operations performed on arrays are:
    - Traversal
    - Searching
    - Sorting

2.2.1 Traversal

- Traversal refers to visiting each element of the array, in a sequential order, to perform an action such as printing or processing.
- The following is a program to traverse an array:

```
#include <stdio.h>
void main() {
   int arr[5];  // Array of 5 integers
   int i;

   // Input elements
   printf("Enter 5 elements of the array:\n");
   for(i = 0; i < 5; i++) {
      scanf("%d", &arr[i]);
   }

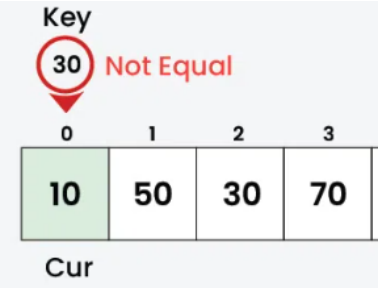
   // Traversing and displaying elements
   printf("The elements in the array are:\n");
   for(i = 0; i < 5; i++) {
      printf("%d ", arr[i]);
   }
}
```

2.2.2 Searching

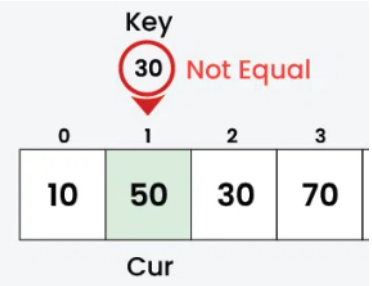
- Searching in an array refers to the process of finding a specific element in a given list of elements.
- The goal is to determine whether the element exists in the array and, if so, find its index (position).
- There are two main types of searching techniques in an array:
    - Linear Search (Sequential Search)
    - Binary Search

2.2.2.1 Linear Search

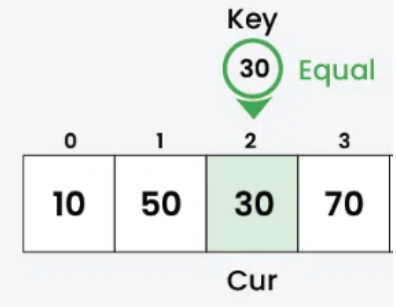
- Linear search, also known as sequential search, is a simple algorithm used to find a specific element within an array or list.
- It works by sequentially checking each element of the list until a match is found or the entire list has been traversed.
- It traverses the array one element at a time and compares each element with the target value.
- If a match is found, it returns the index of the element.
- If the element is not found, the search continues until the end of the array.
- For example: Consider the array arr[] = {10, 50, 30, 70} and key = 30
    a) Compare the key with each element one by one starting from the first element



b) Compare the key with the second element, which is not equal to the key, so move to the next element.



c) Compare the key with the third element. Match is found, therefore stop the search.



- Program to perform Linear Search

```
#include <stdio.h>
void main() {
   int arr[100], n, key, i, found = 0;
   printf("Enter the number of elements: ");
   scanf("%d", &n);

   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
```

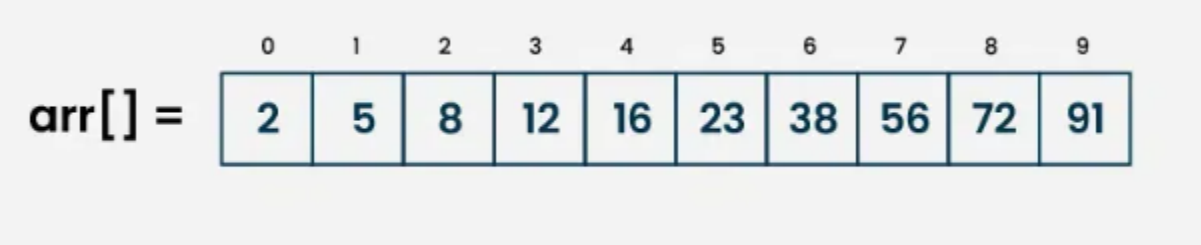
```
    printf("Enter the element to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (arr[i] == key) {
            printf("Element found at position %d\n", i + 1);  // Position is index + 1
            found = 1;
            break;
        }
    }

    if (found == 0) {
        printf("Element not found in the array.\n");
    }
}
```

2.2.2.1 Binary Search

- Binary Search is an efficient searching algorithm that works on sorted arrays.
- It repeatedly divides the search interval in half until the target element is found.
- Binary search is a divide-and-conquer algorithm-It repeatedly **divides** the search space and **conquers** by searching one half.
- Step-by-step algorithm for Binary Search is as follows:
  - Divide the search space into two halves by finding the middle index "*mid*".
  - Compare the middle element of the search space with the *key*.
  - If the *key* is found at the middle element, the process is terminated.
  - If the *key* is not found at the middle element, choose which half will be used as the next search space.
    - If the *key* is smaller than the middle element, then the *left side* is used for the next search. Update high = mid - 1.
    - If the *key* is larger than the middle element, then the *right side* is used for the next search.Update low = mid + 1.
  - This process is continued until the *key* is found or the total search space is exhausted.
- To understand the working of binary search, consider the following illustration:
  - Consider an array **arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}**, and the **key= 23**.
  - Initially the array is as shown below and the key to search is 23.



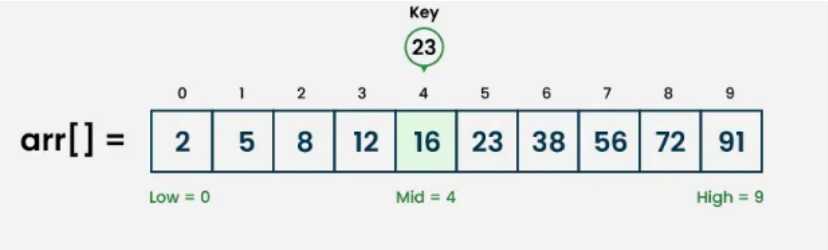
1) Step 1:

Low = 0, High = 9, Mid = (0 + 9) / 2 = 4

arr[4] = 16

Compare key (23) with arr[4] (16):

23 > 16, so search the right half (indices 5–9).



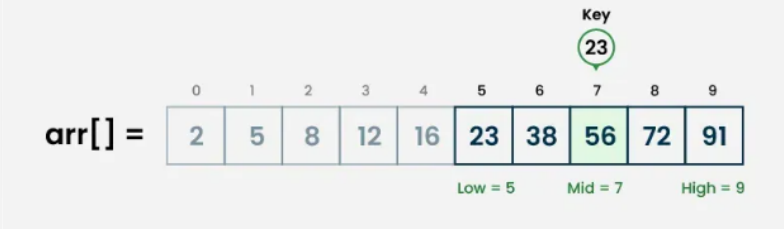
2) Step 2:

Low = Mid+1= 4+1 = 5, High = 9, Mid = (5 + 9) / 2 = 7

arr[7] = 56

Compare key (23) with arr[7] (56):

23 < 56, so search the left half (indices 5–6).



3) Step 3:

Low=5, High = Mid-1= 7-1 = 6, Mid = (5 + 6) / 2 = 5

arr[5] = 23

Compare key (23) with arr[5] (23):

23 = 56, so match found.



- Program:

```
#include <stdio.h>

void main() {
   int arr[20],n, key, low, high, mid,found = 0;
   printf("Enter the number of elements: ");
   scanf("%d", &n);
   printf("Enter the elements in sorted order: ");
   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
   printf("Enter the element to search: ");
   scanf("%d", &key);

   low = 0;
   high = n - 1;

   while (low <= high) {
      mid = (low + high) / 2;
      if (arr[mid] == key) {
         printf("Element %d found at index %d\n", key, mid);
         found = 1;
         break;
      }
      else if (arr[mid] < key) {
         low = mid + 1;
      }
      else {
         high = mid - 1;
      }
   }

   if (found==0) {
      printf("Element %d not found in the array.\n", key);
   }
}
```

Divide and Conquer Method

- Divide and Conquer is a problem-solving strategy where a problem is broken down into smaller subproblems of the same type, solved independently, and then combined to form the final solution.
- A divide-and-conquer algorithm generally has three steps:
    1. Divide → Break the problem into smaller subproblems.
    2. Conquer → Solve the subproblem(s).

3. Combine → Merge the solutions of the subproblems.

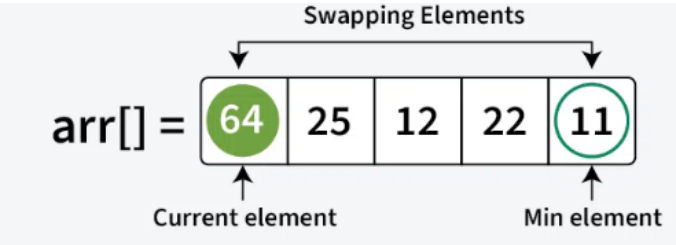
- Examples: Binary Search, Merge Sort, Quick Sort.

2.2.3 Sorting

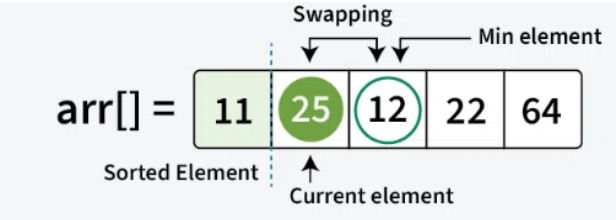
- Sorting is the process of arranging data in a particular order — most commonly ascending (smallest → largest) or descending (largest → smallest).
- Types of Sorting: some of the types of sorting are: bubble sort, selection sort, quick sort etc.

2.2.3.1 Selection Sort

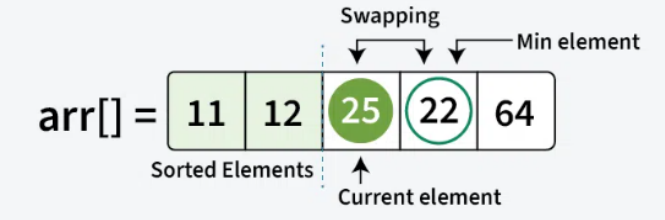
- Selection Sort sorts an array by repeatedly selecting the **smallest** element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.
  1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
  2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
  3. We keep doing this until we get all elements moved to correct position.
- It follows the divide and conquer strategy to sort the array elements.
- Example:
  Consider the array arr[] with elements 64, 25, 12, 22, 11
  1) Initially the sorted array is empty and the unsorted array contains all the elements. Find the smallest element in the array, starting from the element at the index 0. Swap it (11) with the first element (64) in the unsorted array.



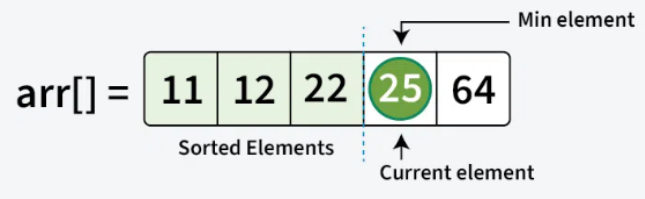
  2) Move to the next element at index 1 (25). Find the smallest element in the unsorted subarray and swap it with the current element.



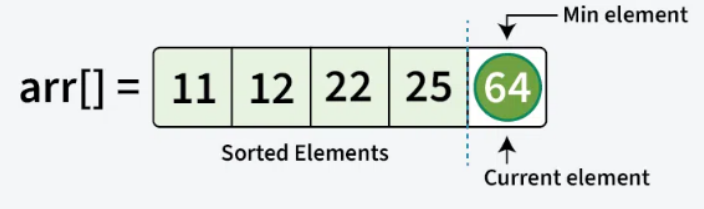
  3) Move to the next element at index 2 (25). Find the smallest element in the unsorted array and swap it with the current element.



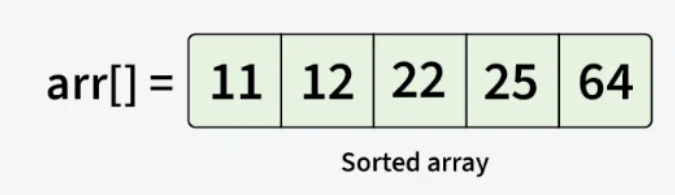
4) Move to the next element at index 3 (25). Find the smallest element in the unsorted array and swap it with the current element.



5) Move to the next element at index 4 (64). Find the smallest element in the unsorted array and swap it with the current element.



6) Here is the sorted array.



- Program:

```
#include <stdio.h>
void main() {
   int n, arr[20], i, j, min, temp;

   printf("Enter the number of elements: ");
   scanf("%d", &n);
   printf("Enter the elements: ");
   for (i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }
   for (i = 0; i < n - 1; i++) {
      min = i;
      for (j = i + 1; j < n; j++) {
         if (arr[j] < arr[min]) {
```

```
                min = j;
            }
        }
        if(min!=j)
        {
            temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

2.2.3.2 Quick Sort

- Quicksort is a divide-and-conquer sorting algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot.
- The sub-arrays are then sorted recursively.
- There are mainly three steps in the algorithm:
  1. **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
  2. **Partition the Array:** Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
  3. **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot). The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.
- When the last element is chosen as the pivot, the process is as follows:
  - **Choose Pivot:** Select the last element (arr[high]) of the array (or sub-array) as the *pivot*.
  - **Partitioning:**
    - *start* begins at *low* (left side).
    - *end* begins at *high - 1* (just before pivot).
    - While *start* <= *end*:
      1. Move *start* right until you find an element greater than *pivot*.
      2. Move *end* left until you find an element less than or equal to *pivot*.
      3. If *start* < *end*, swap arr[*start*] and arr[*end*].

    - This ensures:
      - All elements smaller than *pivot* move left.

        - All elements greater than *pivot* move right.
    - **Place *pivot* in Correct Position**
        - After the loop ends, swap arr[*start*] with the *pivot* (arr[high]).
        - Now *pivot* is in its final sorted position.
        - Return *start* as partition index.

    - **Recursion:**
        - Apply Quick Sort to:
            - Left sub-array : from *low* to *pivot - 1*
            - Right sub-array: from *pivot + 1* to *high*

- Example: Consider the Array: arr[]={10, 7, 8, 9, 1, 5}, Pivot = 5, start=0, end=4
- Iteration:
    - arr[start]=10 > 5 (stop)
    - arr[end]=1 <= 5 (stop)
    - Swap arr[start] and arr[end] → {1, 7, 8, 9, 10, 5}
    - arr[start]=1<5; increment start
    - arr[start]=7>5 (stop)
    - arr[end]=10>5; decrement end
    - arr[end]=9>5; decrement end
    - arr[end]=8>5; decrement end
    - arr[end]=7>5; decrement end
    - Now start=1, end=1 → loop ends.
    - Swap arr[start] and pivot → {1, 5, 8, 9, 10, 7}
    - Pivot (5) is correctly placed at index 1.
    - Then recursion continues on subarrays {1} (sorted) and {8, 9, 10, 7}.
    - Final sorted array will be {1, 5, 7, 8, 9, 10}

- Program:

```
#include <stdio.h>
int partition(int arr[], int low, int high) {
   int pivot = arr[high];
   int start = low;
   int end = high - 1;
   int temp;

   while (start <= end) {
      while (start <= end && arr[start] <= pivot)
         start++;

      while (start <= end && arr[end] > pivot)
         end--;
```

```
        if (start < end) {
            temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
        }
    }

    temp = arr[start];
    arr[start] = arr[high];
    arr[high] = temp;

    return start;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void main() {
    int n, arr[50], i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

2.3 Two Dimensional Arrays

- A two-dimensional array in C is a collection of elements of the same data type arranged in a tabular format, similar to a matrix or a table with rows and columns.
- It is an "array of arrays."
- Declaration: A two-dimensional array is declared using two sets of square brackets, one for the number of rows and one for the number of columns.
- Syntax:

```
data_type array_name[row_size][column_size];
```

- Example:

```
int matrix[3][4];    // Declares a 2D array named 'matrix' with 3 rows and 4 columns
```

- In C, arrays are 0-indexed, so the row number ranges from 0 to (m-1) and the column number ranges from 0 to (n-1).

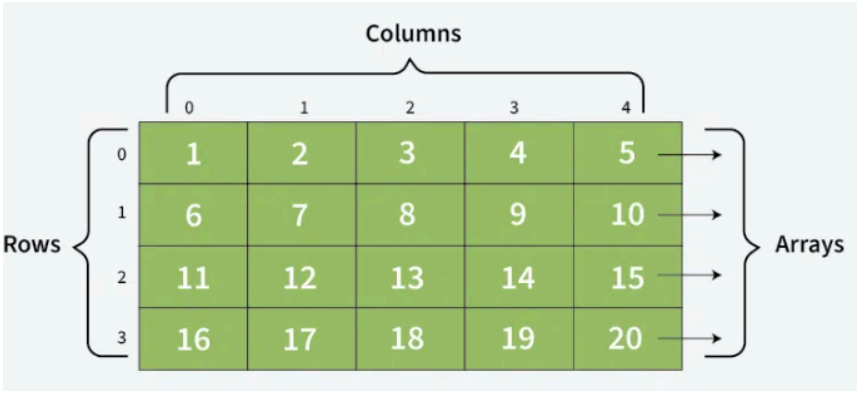


Fig 2.1 A 2D Array with 4 rows and 5 columns

- Initialization: can be done in the following ways
  - int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };
  - int arr[2][3] = {1, 2, 3, 4, 5, 6};
  - int arr[][3] = { 1, 2, 3, 4, 5, 6 }; // Row size can be omitted but column size should be given
- Accessing Elements
  - Elements in a two-dimensional array are accessed using two indices: one for the row and one for the column.

    ```
    array_name[row_index][column_index];
    ```
  - Example:

    ```
    int value = matrix[0][1]; // Accesses the element at row 0, column 1
    ```
- Input Elements into a 2D Array:

- To accept input from the user and store it in a two-dimensional array, nested for loops are used.
- The outer loop iterates through each row, and the inner loop iterates through each column within that specific row.
- Example:

```
#include <stdio.h>
void main() {
   int rows, cols;
   int arr[10][10];   // maximum size
   int i, j;

   printf("Enter number of rows and columns: ");
   scanf("%d %d", &rows, &cols);     // Read matrix size
   printf("Enter elements of the matrix:\n");
   for(i = 0; i < rows; i++) {
      for(j = 0; j < cols; j++) {
         scanf("%d", &arr[i][j]);       //Input matrix elements
      }
   }
}
```

- Output Elements of 2D Array:

```
#include <stdio.h>

void main() {
   int rows = 2, cols = 3;
   int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };
   int i, j;
   printf("The matrix is:\n");
   for(i = 0; i < rows; i++) {
      for(j = 0; j < cols; j++) {
         printf("%d ", arr[i][j]);          }
      printf("\n");
   }
}
```

Output:
The matrix is:
1 2 3
4 5 6

2.3.1 Memory Representation of 2D Arrays

- A 2D array is stored in memory as a sequence of contiguous memory locations, using either row-major order (elements of the first row, then the second row, etc., are stored consecutively) or column-major order (elements of the first column, then the second column, etc., are stored consecutively).

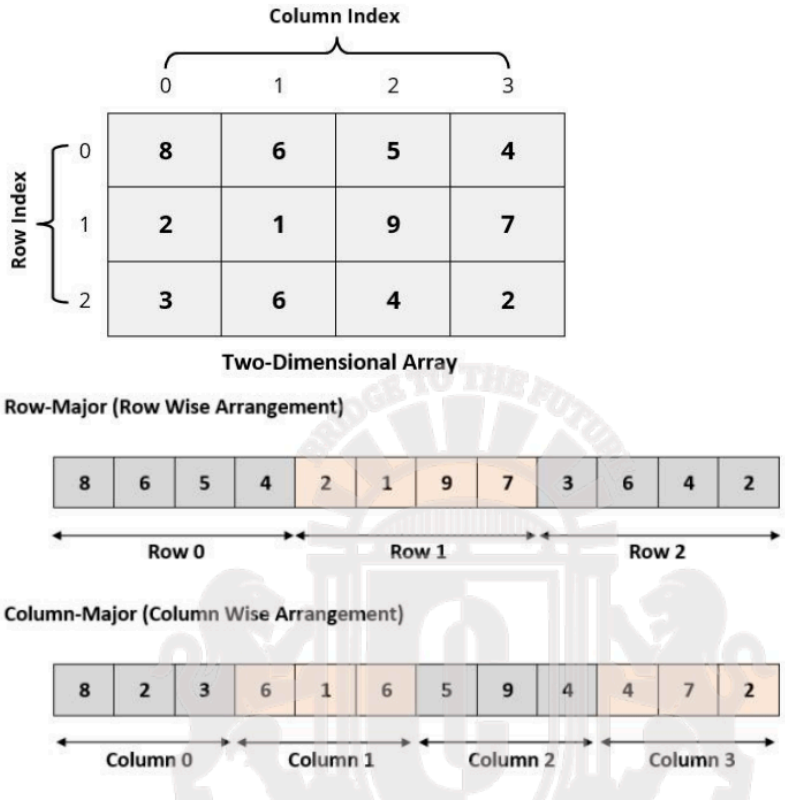


Fig. 2.2 Row-Major and Column-Major Representation

2.3.2. Program to Add Two Matrices

```
#include <stdio.h>
void main() {
   int rows, cols;
   int A[10][10], B[10][10], C[10][10];
   int i, j;

   // input matrix size
   printf("Enter number of rows and columns: ");
   scanf("%d %d", &rows, &cols);

   //  Input first matrix
   printf("Enter elements of first matrix:\n");
   for(i = 0; i < rows; i++) {
      for(j = 0; j < cols; j++) {
```

```
            scanf("%d", &A[i][j]);
        }
    }

    // Input second matrix
    printf("Enter elements of second matrix:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            scanf("%d", &B[i][j]);
        }
    }

    // Add matrices
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    // Display result
    printf("\nResultant Matrix (A + B):\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
}
```

2.4 Passing Arrays to Functions

- Passing arrays to functions involves providing a reference to the array's data, allowing the function to access and modify the original array's elements.
- In C, arrays are always passed by reference (as pointers).That means when you pass an array to a function, you are really passing the address of its first element.
- Passing a 1D Array to a Function:
    - When you pass an array to a function, you're actually passing the *base address* (a pointer to the first element).
    - So changes made inside the function affect the original array.
    - Example:

```
#include <stdio.h>
void printArray(int arr[], int size) {
    int i;
    for(i = 0; i < size; i++) {
```

```
        printf("%d ", arr[i]);
    }
  }

  void main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int size = 5;
    printArray(numbers, size);  // passing array
  }
```

- Passing a 2D Array to a Function:
  - For 2D arrays, we must specify the number of columns (but not necessarily the number of rows).
  - This is because memory is stored row by row, and the compiler needs the column size to calculate the correct address.
  - Example:

```
#include <stdio.h>

// Function to print a 2D array
void printMatrix(int arr[][10], int rows, int cols) {
    int i, j;
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}

void main() {
    int rows, cols;
    int matrix[10][10];
    int i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Input matrix elements
    printf("Enter matrix elements:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }
```

```
    // Output using function
    printf("\nThe matrix is:\n");
    printMatrix(matrix, rows, cols); // Passing two dimensional array
}
```

2.5 Strings

- In C language, there is no built-in string data type.
- Instead, a string is represented as an array of characters ending with a special null character '\0'.
- A string is a sequence of characters + '\0'.

2.5.1 Representation of Strings

- String as Character Array

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

  - Here, 'H', 'e', 'l', 'l', 'o' are stored in the array.
  - The last character '\0' marks the **end of the string**.
- String Literal

```
char str[] = "Hello";
```

  - This is equivalent to the previous example, because the compiler automatically adds the **null character** at the end.

2.5.2 String Output

- We can print strings using:
  - printf with %s

```
char str[] = "Hello World";
 printf("%s\n", str);  // %s is format specifier for strings
```

  - puts

```
puts(str);
```

    - Prints the string followed by a newline automatically.

2.5.3 String Input

- There are multiple ways to input a string:

a) Using scanf

  - Example:

```
 char name[50];
 printf("Enter your name: ");
scanf("%s", name);   // reads until space
```

- Limitation scanf("%s", name) stops at space, so it won't read full names with spaces.

b) Using gets (deprecated)

- Syntax:

```
char str[SIZE];
gets(str);
```

- Example:

```
char name[20];
gets(name);
```

- Reads entire line including spaces.

c) Using fgets ()

- Syntax:

char *fgets(char *str, int size, FILE *stream);
str - character array to store input
size → maximum characters to read
stream → usually stdin for keyboard input

- Example:

```
char name[20];
fgets(name, sizeof(name), stdin);
```

d) scanf("%[^\n]")

- This is a scanset format specifier.
- %[^\n] → read input until a newline is encountered.
- Syntax:

```
scanf("%[^\n]", str);
```

- Example:

```
char sentence[50];
scanf("%[^\n]", sentence);
```

2.5.4 String Operations

- C language provides various built-in functions that can be used for various operations and manipulations on strings.
- These string functions make it easier to perform tasks such as string copy, concatenation, comparison, length, etc.
- The **<string.h>** header file contains these string functions.
- Some common string operations and their corresponding functions are:
  1) Finding String Length - strlen()
     - The **strlen()** function is used to find the length of a string.
     - It returns the number of characters in a string, excluding the null terminator ('\0').

- Syntax:

```
strlen(string);
```

- Example:

```
#include <stdio.h>
#include <string.h>
void main() {
   char str[] = "Hello World";
   printf("Length = %d", strlen(str));  //Outputs 11
 }
```

2) Copying Strings: strcpy()
   - Copies the content of source string into destination string.
   - Syntax:

```
strcpy(dest,source)
```

   - Example:

```
#include <stdio.h>
#include <string.h>
void main() {
   char source[] = "Programming";
   char dest[20];
   strcpy(dest, source);
   printf("Destination: %s", dest);  //Outputs Programming
 }
```

3) Concatenating Strings: strcat()
   - Concatenates (joins) two strings.
   - Appends the source string to the end of the destination string.
   - Syntax:

```
strcat(dest,src);
```

   - Example:

```
#include <stdio.h>
#include <string.h>
void main() {
   char str1[50] = "Good ";
   char str2[] = "Morning";
   strcat(str1, str2);
   printf("Concatenated String: %s", str1);  //Outputs Good Morning
 }
```

4) Comparing Strings: strcmp()
   - Compare two strings **lexicographically** (like dictionary order).
   - Returns either of the following values:
     - 0 → if both strings are equal
     - <0 (negative value) → if str1 < str2
     - >0 (positive value) → if str1 > str2

- Syntax:

  strcmp(string1, string2);

- Example:

```
#include <stdio.h>
#include <string.h>
void main() {
   char a[] = "Apple";
   char b[] = "Banana";
   int result = strcmp(a, b);
   if (result == 0)
      printf("Strings are equal");
   else if (result < 0)
      printf("First string is smaller");
   else
      printf("First string is greater");
}
```

Output:

First string is smaller